

[Pro](#) [Teams](#) [Pricing](#) [Documentation](#)**npm**[Sign Up](#)[Sign In](#)[Search](#)

simple-ssh TS

1.1.1 • [Public](#) • Published 5 months ago[Readme](#)[Code](#) Beta[2 Dependencies](#)[55 Dependents](#)[21 Versions](#)

Note: *I really don't work on this project anymore. It's still fully functional, I'm just not interested in working on it anymore. If you post an issue, I might respond and I might not. If you submit a pull request, I'll most likely accept it. Hopefully it continues to help people around the world.* 🌍

simple-ssh

A wrapper for the [ssh2 client module](#) by [Brian White](#) which makes it easier to run a sequence of commands over SSH.

Requirements

- [node.js](#) -- v0.8.7 or newer

Install

```
npm install simple-ssh
```

Examples

- Echoing out a users `PATH` :

```
var SSH = require('simple-ssh');

var ssh = new SSH({
  host: 'localhost',
  user: 'username',
  pass: 'password'
});

ssh.exec('echo $PATH', {
  out: function(stdout) {
    console.log(stdout);
  }
}).start();

/** Using the `args` options instead */
ssh.exec('echo', {
  args: ['$PATH'],
  out: function(stdout) {
    console.log(stdout);
  }
}).start();
```

- Connecting with the active SSH Agent with Agent Forwarding

```
var ssh = new ssh({
  host: 'localhost',
```

```
    user: 'username',  
    agent: process.env.SSH_AUTH_SOCK,  
    agentForward: true  
  })
```

- Capturing error output:

```
ssh.exec('this-does-not-exist', {  
  err: function(stderr) {  
    console.log(stderr); // this-does-not-exist: command not found  
  }  
}).start();
```

- Capturing error codes:

```
ssh.exec('exit 69', {  
  exit: function(code) {  
    console.log(code); // 69  
  }  
}).start();
```

- Sending data to stdin:

```
ssh.exec('cat > /path/to/remote/file', {  
  in: fs.readFileSync('/path/to/local/file')  
}).start();
```

- Chaining commands together:

```
ssh  
  .exec('echo "Node.js"', {  
    out: console.log.bind(console)  
  })
```

```
.exec('echo "is"', {
  out: console.log.bind(console)
})
.exec('echo "awesome!"', {
  out: console.log.bind(console)
})
.start();

// Output:
// Node.js
// is
// awesome!
```

- Get the number of commands:

```
ssh
  .exec('exit 1')
  .exec('exit 2')
  .exec('exit 3');

console.log(ssh.count()); // 3
```

- Running a command using `sudo`

```
ssh.exec('sudo echo "Pseudo-sudo"', {
  pty: true,
  out: console.log.bind(console)
}).start();
```

- Resetting a connection and the commands

```
// Echos out any messages the user sent in if 10 or more have been que
var msgInterval = setInterval(function() {
```

```
    if (ssh.count() > 10) {
      ssh.start();
    }
  }, 1000);

socket.on('message', function(msg) {
  // If a 'reset' message is received, clear previous messages
  if (msg === 'reset') {
    ssh.reset(function(err) {
      if (err) {
        throw err;
      }

      ssh.exec('echo "reset"');
    });
  } else {
    ssh.exec('echo "' + msg + '"');
  }
});
```

- Listening for additional events

```
ssh.on('error', function(err) {
  console.log('Oops, something went wrong.');
```

```
  console.log(err);
  ssh.end();
});
```

- Event handlers can be chained as well

```
ssh
  .on('error', onSSHError)
  .on('ready', onSSHReady);
```

API

Functions

- **Constructor**([*config*])
 - **config** { *Object* }:
 - **config.host** { *String* } : Hostname
 - **config.port** { *Number* } : Port number (default: 22)
 - **config.user** { *String* } : Username
 - **config.pass** { *String* } : Password
 - **config.timeout** { *Number* } : Connection timeout in milliseconds (default: 10000)
 - **config.key** { *String* } : SSH key
 - **config.passphrase** { *String* } : Passphrase
 - **config.baseDir** { *String* } : Base directory. If this is set, each command will be preceded by `cd ${this.baseDir}`
 - **config.agent** { *String* } : Connects with the given SSH agent. If this is set, no need to specify a private key or password.
 - **config.agentForward** { *Boolean* } : Set to true to connect with agent forwarding.
- **exec**(*command*, [*options*]) : **Adds a command to the stack**
 - **command** { *String* } : Command to be executed
 - **options** { *Object* } :
 - **options.args** { *String*[] } : Additional command line arguments (default: null)
 - **options.in** { *String* } : Input to be sent to `stdin`
 - **options.out** { *Function*(*stdout*) } : `stdout` handler
 - **stdout** { *String* } : Output streamed through `stdout`
 - **options.err** { *Function*(*stderr*) } : `stderr` handler
 - **stderr** { *String* } : Output streamed through `stderr`
 - **options.exit** { *Function*(*code*, *stdout*, *stderr*) } : Exit handler
 - **code** { *Number* } : Exit code
 - **stdout** { *String* } : All of the standard output concatenated together
 - **stderr** { *String* } : All of the error output concatenated together
 - **options.pty** { *Boolean* } : Allocates a pseudo-tty, useful for command which require `sudo` (default: false)
- **on**(*event*, *callback*) : **Add a listener for the specified event** (Courtesy of @alexjab)

- **event** { *String* }: Event to listen to
- **callback** { *Function* }: Executed on the event
- **start**([*options*]): **Starts executing the commands**
 - **options** { *Object* }:
 - **options.success** { *Function()* }: Called on successful connection
 - **options.fail** { *Function(err)* }: Called if the connection failed
 - **err** { *Error* }: Error information
- **reset**([*callback*]): **Clears the command queue and resets the current connection**
 - **callback** { *Function(err)* }: Called when the connection has been successfully reset
 - **err** { *Error* }: Error information
- **end()**: **Ends the SSH session** (this is automatically called at the end of a command queue).

Properties

- **host** { *String* }: Host to connect to
- **port** { *Number* }: Port to connect through (default: 22)
- **user** { *String* }: User name
- **pass** { *String* }: Password
- **timeout** { *Number* }: Connection timeout in milliseconds (default: 10000)
- **key** { *String* }: SSH key
- **baseDir** { *String* }: If set, will change directory to `baseDir` before each command

Flow Control

Sometimes you may find yourself needing to change which commands are executed. The flow can be changed by returning `false` from an `exit` handler.

Note: This only works if `false` is explicitly returned. "Falsy" values are not sufficient (since `undefined` is implicitly returned and it's "falsy").

- Ending prematurely:

```
ssh
  .exec('pwd', {
    exit: function() {
      return false;
    }
  })
})
```

```
.exec('echo "Not executed"')  
.start();
```

- Running a new queue of commands:

```
ssh  
  .exec('exit', {  
    args: [ Math.round(Math.random()) ],  
    exit: function(code) {  
      if (code === 1) {  
        // Setup the new command queue  
        ssh.exec('echo "new queue"');  
        return false;  
      }  
    }  
  })  
  .exec('exit 0', {  
    exit: function() {  
      console.log('Previous command did not return false');  
    }  
  })  
  .start();
```

Keywords

ssh ssh2

Install

```
> npm i simple-ssh
```


Repository

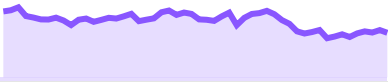
 github.com/MCluck90/simple-ssh

Homepage

 github.com/MCluck90/simple-ssh#readme

Weekly Downloads

3,014



Version	License
1.1.1	none

Unpacked Size	Total Files
31.2 kB	9

Issues	Pull Requests
13	0

Last publish
5 months ago

Collaborators



[Try on RunKit](#)

[Report malware](#)



Support

[Help](#)

[Advisories](#)

[Status](#)

[Contact npm](#)

Company

[About](#)

[Blog](#)

[Press](#)

Terms & Policies

[Policies](#)

[Terms of Use](#)

[Code of Conduct](#)

[Privacy](#)